

**A.Y: 2026 – 27**  
**RESEARCH METHODOLOGY ISE-I**  
**CASE STUDY**

**DOCS: PROGRAMMING LABORATORY**  
**MANAGEMENT AND PRACTICAL ASSESSMENT**  
**PLATFORM**

Literature Review, Research Gap and AI Tools

**Student Information**

| <b>Student Name</b>     | <b>Roll No.</b> | <b>Division</b> |
|-------------------------|-----------------|-----------------|
| Dhruv Dipak Dhamanaskar | 105             | A               |
| Om Milind Haval         | 106             | A               |
| Chirayu Pravin Maske    | 107             | A               |
| Siddhi Nitin Chavan     | 108             | A               |

**Project Guide**

Dr. G. V. Patil

**Project Domain**

Data Science, Artificial Intelligence and Programming Education

## Table of Contents

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| 1. Problem Statement                                                           | 3  |
| 2. Objectives                                                                  | 4  |
| 3. Literature Review                                                           | 5  |
| 3.1 Research Paper 1: Web-CAT                                                  | 5  |
| 3.2 Research Paper 2: Automatic Assessment of Programming Assignments          | 5  |
| 3.3 Research Paper 3: Automated Feedback Generation                            | 5  |
| 3.4 Research Paper 4: Automated Grading and Feedback Tools – Systematic Review | 6  |
| 3.5 Research Paper 5: Large Language Models (GPT) for Programming Feedback     | 6  |
| 3.6 Research Paper 6: LLMs for Formative Programming Feedback                  | 6  |
| 4. Literature Review Table                                                     | 7  |
| 4.1 Paper 1: Web-CAT                                                           | 7  |
| 4.2 Paper 2: Automatic Assessment Review                                       | 8  |
| 4.3 Paper 3: Automated Feedback Generation                                     | 9  |
| 4.4 Paper 4: Systematic Review of Grading and Feedback Tools                   | 10 |
| 4.5 Paper 5: GPT Feedback                                                      | 11 |
| 4.6 Paper 6: LLM Formative Feedback                                            | 12 |
| 5. Research Gap                                                                | 13 |
| 6. AI Tools Used for Research Support                                          | 14 |
| 7. References                                                                  | 15 |

# 1. Problem Statement

The Final Year Project research topic, DOCS (Departmental Online Coding System), was selected and finalized in consultation with the Project Guide, Dr. G. V. Patil, as a platform for managing programming-oriented practical laboratory activities within an academic department.

In many undergraduate computer science departments, practical laboratory work is managed through a combination of physical lab manuals, faculty instructions, handwritten journals, separate programming environments, spreadsheets, and manual evaluation. This fragmented workflow makes it difficult to maintain a consistent and structured practical-learning environment across multiple batches.

Programming-based laboratory assessment introduces additional challenges. Faculty members must prepare experiment-wise questions, evaluate multiple student submissions, provide feedback, track attempts, and maintain marks. Manual evaluation can become repetitive and time-consuming, while differences in marking practices can introduce inconsistency. Research on automated programming assessment similarly identifies manual grading as repetitive and time-intensive and highlights the value of automated test-case-based assessment and formative feedback (Messer et al., 2024).

Students also need opportunities to practice before and during scheduled practical sessions. A useful system therefore needs to distinguish between practice activity and assessed activity, retain multiple attempts, evaluate code against test cases, and provide feedback that helps students improve rather than merely indicate whether an answer is correct.

Another gap is the lack of consolidated learning analytics. Student attempts, submission outcomes, completion time, difficulty progression, pass/fail rates and non-submission information can provide useful insights to faculty and administrators. Research on learning analytics in virtual laboratories found that much of the existing analytics work relies on student activity/log data and concluded that the field remains fragmented and has not fully exploited analytics to support learning and teaching.

The emergence of generative AI introduces another opportunity. AI can support students while programming, but unrestricted AI assistance may improve task performance without necessarily improving conceptual learning. Recent research therefore suggests that scaffolded, hint-based AI assistance may be more educationally appropriate than simply providing complete solutions.

Therefore, DOCS (Departmental Online Coding System) is proposed as a centralised web-based laboratory platform that integrates academic organisation, practical experiment management, programming assessment, multiple attempts, assignments, resources, plagiarism checking, marks, analytics, and AI-assisted learning into a single system.

## 2. Objectives

The primary objective of DOCS is to develop a flexible digital platform for managing programming-oriented practical laboratory activities within an academic department. Specific objectives include:

- **Centralize laboratory management:** Maintain subjects, experiments, practical questions, notes and laboratory manuals in one system.
- **Support flexible experiment structures:** Allow faculty to define any number of experiments for each subject rather than hardcoding a fixed number.
- **Implement structured practical sessions:** Allow students to practice experiments before the scheduled assessment period while counting marks only during the designated practical session.
- **Provide automated programming assessment:** Execute student code against faculty-defined test cases and calculate pass/fail results and performance.
- **Support multiple attempts:** Preserve all student attempts and allow students to improve their solutions.
- **Provide difficulty-based learning:** Organize questions into configurable difficulty levels such as Easy, Medium and Hard.
- **Support assignment management:** Allow faculty to create optional assignments with deadlines, reminders and configurable late penalties.
- **Provide consistent evaluation:** Use common subject-wise experiment/question structures for students within the relevant batches.
- **Support plagiarism checking:** Allow faculty to initiate plagiarism analysis after the practical session.
- **Provide analytics and leaderboards:** Analyze submission counts, pass/fail percentages, complexity, completion, difficulty progression and non-submission.
- **Provide AI-assisted learning:** Allow students to ask AI for assistance and receive improvement-oriented guidance.
- **Maintain historical academic records:** Preserve records across academic sessions such as 2026–27, 2027–28 and subsequent years.

### 3. Literature Review

Six research papers relevant to the DOCS project topic were downloaded and studied from reputed research databases, covering automated programming assessment, automated feedback, and AI-assisted programming education.

#### 3.1 Research Paper 1: Web-CAT (Edwards and Pérez-Quñones, 2008)

Edwards and Pérez-Quñones presented Web-CAT as a web-based system for automatically grading programming assignments. The work demonstrates how instructor-defined assessment strategies and automated testing can provide scalable programming assessment. Its relevance to DOCS is the automated evaluation component, where programming submissions can be checked consistently against defined requirements.

The system primarily addresses programming assignment assessment rather than the complete department-level practical workflow. DOCS extends the assessment concept into scheduled practical sessions, retained multiple attempts, academic structure, AI-assisted improvement, analytics and faculty-controlled post-lab processes.

#### 3.2 Research Paper 2: Automatic Assessment of Programming Assignments (Ihantola et al., 2010)

Ihantola et al. reviewed recent systems for automatic assessment of programming assignments and discussed testing, resubmission, security and pedagogical approaches. The review provides a foundation for understanding the design space of automated programming assessment.

The work establishes that automated assessment is a mature area with multiple approaches, but it does not itself provide a unified department-oriented practical-session environment. DOCS uses this literature as the basis for its objective test-case-based evaluation model.

#### 3.3 Research Paper 3: Automated Feedback Generation (Singh et al., 2013)

Singh, Gulwani and Solar-Lezama studied automated feedback generation for introductory programming assignments using reference implementations and error models. The study demonstrates the possibility of automatically identifying issues in student submissions and generating corrective guidance.

The study supports giving students actionable feedback after unsuccessful submissions. However, feedback generation is treated separately from practical-session management and academic administration. DOCS incorporates feedback into a retained multi-attempt workflow.

### 3.4 Research Paper 4: Automated Grading and Feedback Tools – Systematic Review (Messer et al., 2024)

Messer et al. conducted a systematic review of automated grading and feedback tools for programming education, covering 121 research papers. The review identifies dynamic unit testing and static analysis among important approaches and examines the state of automated feedback and rapid resubmission.

The review demonstrates the maturity of automated assessment while identifying continuing limitations such as the concentration of feedback on correctness and limited availability of evaluation datasets. DOCS is positioned as an integrated practical workflow around these established assessment capabilities.

### 3.5 Research Paper 5: Large Language Models (GPT) for Programming Feedback (Pankiewicz and Baker, 2023)

Pankiewicz and Baker investigated GPT-based personalized programming feedback. The study explored generated hints and reported generally positive student perceptions while also identifying the possibility of student over-reliance on AI.

The findings support AI as a formative support mechanism rather than the sole evaluator. DOCS follows this principle by using AI for feedback and betterment while official marks remain based on deterministic test cases and faculty-defined assessment rules.

### 3.6 Research Paper 6: LLMs for Formative Programming Feedback (Kiesler et al., 2023)

Kiesler, Lohr and Keuning investigated the potential of Large Language Models for generating formative feedback on programming solutions. The study is relevant to the growing use of generative AI in programming education and highlights the need to consider the quality and reliability of generated feedback.

The research supports AI-assisted formative feedback in DOCS while reinforcing the need for objective and human controls. DOCS therefore separates AI assistance from the authoritative assessment mechanism.

## 4. Literature Review Table

Detailed comparative assessment of the selected research papers, presented as individual literature review tables covering Research Paper Title, Authors, Publication Year, Research Methodology/Technique Used, Dataset/Data Source, Key Findings, Limitations, and Identified Research Gap:

### 4.1 Paper 1: Web-CAT

|                                               |                                                                                                                                                      |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Research Paper Title:</b>                  | <a href="#">Web-CAT: Automatically Grading Programming Assignments</a>                                                                               |
| <b>Authors:</b>                               | S. H. Edwards and M. A. Pérez-Quñones                                                                                                                |
| <b>Publication Year:</b>                      | 2008                                                                                                                                                 |
| <b>Research Methodology / Technique Used:</b> | Web-based customizable automated programming assessment using instructor-defined tests and assessment strategies.                                    |
| <b>Dataset / Data Source:</b>                 | Programming assignments and student submissions.                                                                                                     |
| <b>Key Findings:</b>                          | Demonstrates scalable and configurable automated grading for programming assignments.                                                                |
| <b>Limitations:</b>                           | Primarily focused on programming assessment rather than the complete academic and practical-session workflow.                                        |
| <b>Identified Research Gap:</b>               | Supports DOCS automated test-case evaluation; DOCS extends this into timed practical sessions, multiple attempts, analytics and academic management. |

## 4.2 Paper 2: Automatic Assessment Review

|                                               |                                                                                                                           |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Research Paper Title:</b>                  | <a href="#">Review of Recent Systems for Automatic Assessment of Programming Assignments</a>                              |
| <b>Authors:</b>                               | P. Ihanola, T. Ahoniemi, V. Karavirta and O. Seppälä                                                                      |
| <b>Publication Year:</b>                      | 2010                                                                                                                      |
| <b>Research Methodology / Technique Used:</b> | Review of automated programming-assessment systems, including testing, resubmission, security and pedagogical approaches. |
| <b>Dataset / Data Source:</b>                 | Published research and system literature.                                                                                 |
| <b>Key Findings:</b>                          | Shows the breadth of approaches used for automatic programming assessment.                                                |
| <b>Limitations:</b>                           | The review focuses on assessment systems rather than a single integrated practical-laboratory platform.                   |
| <b>Identified Research Gap:</b>               | Provides the research foundation for DOCS automated assessment and helps position the project within existing systems.    |



### 4.3 Paper 3: Automated Feedback Generation

|                                               |                                                                                        |
|-----------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Research Paper Title:</b>                  | <a href="#">Automated Feedback Generation for Introductory Programming Assignments</a> |
| <b>Authors:</b>                               | R. Singh, S. Gulwani and A. Solar-Lezama                                               |
| <b>Publication Year:</b>                      | 2013                                                                                   |
| <b>Research Methodology / Technique Used:</b> | Automated correction and feedback using reference implementations and error models.    |
| <b>Dataset / Data Source:</b>                 | Real student programming attempts.                                                     |
| <b>Key Findings:</b>                          | Shows the potential of automated feedback to address errors in student submissions.    |
| <b>Limitations:</b>                           | Focused on feedback generation rather than complete laboratory workflow management.    |
| <b>Identified Research Gap:</b>               | Supports DOCS's iterative feedback and multiple-attempt model.                         |

#### 4.4 Paper 4: Systematic Review of Grading and Feedback Tools

|                                               |                                                                                                                                     |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Research Paper Title:</b>                  | <a href="#">Automated Grading and Feedback Tools for Programming Education: A Systematic Review</a>                                 |
| <b>Authors:</b>                               | M. Messer, N. C. C. Brown, M. Kölling and M. Shi                                                                                    |
| <b>Publication Year:</b>                      | 2024                                                                                                                                |
| <b>Research Methodology / Technique Used:</b> | Systematic review of 121 papers covering automated grading and feedback approaches.                                                 |
| <b>Dataset / Data Source:</b>                 | 121 research papers from the reviewed period.                                                                                       |
| <b>Key Findings:</b>                          | Identifies dynamic unit testing and static analysis as important assessment approaches and reviews the state of automated feedback. |
| <b>Limitations:</b>                           | Many systems remain focused on correctness; evaluation datasets are often unavailable.                                              |
| <b>Identified Research Gap:</b>               | Shows maturity of individual assessment tools while supporting investigation of broader workflow integration.                       |

#### 4.5 Paper 5: GPT Feedback

|                                               |                                                                                                                         |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Research Paper Title:</b>                  | <a href="#">Large Language Models (GPT) for Automating Feedback on Programming Assignments</a>                          |
| <b>Authors:</b>                               | M. Pankiewicz and R. S. Baker                                                                                           |
| <b>Publication Year:</b>                      | 2023                                                                                                                    |
| <b>Research Methodology / Technique Used:</b> | GPT-based personalized programming feedback and hints.                                                                  |
| <b>Dataset / Data Source:</b>                 | Student programming assignments and attempts.                                                                           |
| <b>Key Findings:</b>                          | Students generally responded positively to generated hints in the studied setting.                                      |
| <b>Limitations:</b>                           | AI feedback can lead to over-reliance and effectiveness can vary across tasks.                                          |
| <b>Identified Research Gap:</b>               | Supports AI-assisted improvement in DOCS while motivating a strict separation between AI feedback and official grading. |

#### 4.6 Paper 6: LLM Formative Feedback

|                                               |                                                                                                             |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Research Paper Title:</b>                  | <a href="#">Exploring the Potential of Large Language Models to Generate Formative Programming Feedback</a> |
| <b>Authors:</b>                               | N. Kiesler, D. Lohr and H. Keuning                                                                          |
| <b>Publication Year:</b>                      | 2023                                                                                                        |
| <b>Research Methodology / Technique Used:</b> | Evaluation of LLM-generated formative feedback for programming solutions.                                   |
| <b>Dataset / Data Source:</b>                 | Programming solutions and programming tasks.                                                                |
| <b>Key Findings:</b>                          | Shows potential for LLMs to provide formative programming guidance.                                         |
| <b>Limitations:</b>                           | Generated feedback may be misleading and requires validation.                                               |
| <b>Identified Research Gap:</b>               | Supports controlled AI feedback in DOCS rather than autonomous AI grading.                                  |

## 5. Research Gap

The selected literature establishes that automated programming assessment and AI-assisted feedback are important components of programming education. However, these capabilities are commonly investigated or implemented as separate systems or individual functions rather than as an integrated departmental practical workflow. A systematic analysis of the selected literature reveals four critical research gaps that motivate the proposed system:

- **Gap 1 – Fragmented laboratory workflow:** Existing research frequently focuses on a specific component such as automated programming assessment or automated feedback rather than an integrated departmental practical workflow.
- **Gap 2 – Limited academic administration integration:** Automated programming assessment systems generally concentrate on code submission and evaluation rather than the complete chain from department, study year, division, and batch through subject, experiment, practical session, student, attempt, submission, marks, and analytics.
- **Gap 3 – Limited combination of assessment and analytics:** Learning analytics research demonstrates the potential of activity data but also identifies the field as fragmented and exploratory. DOCS proposes capturing assessment activity as a first-class part of the laboratory workflow.
- **Gap 4 – AI must support learning, not merely generate answers:** Recent research shows that unrestricted AI assistance can improve task performance without necessarily improving conceptual learning, while scaffolded approaches show more promising learning-oriented behaviour. DOCS should therefore use AI primarily for hints, error explanations, conceptual guidance, and betterment suggestions rather than simply generating complete solutions.

## 6. AI Tools Used for Research Support

As permitted by the ISE-I guidelines, AI-based tools were used strictly as research-support aids to assist in understanding research papers, summarising content, comparing studies, and identifying potential research gaps:

- **ChatGPT & Google Gemini:** Used as analytical research-support tools to assist in understanding research papers, summarising methodologies and findings, comparing limitations, organizing the literature review and developing a preliminary research-gap synthesis.
- **Grammarly:** Used as an editorial tool for proofreading, syntax refinement, vocabulary consistency and readability improvement.

All AI-generated outputs were treated strictly as preliminary drafts. Every paper-specific finding, empirical figure, and bibliographic detail was critically verified and validated against the original research papers before inclusion in this case study. AI tools were used only as a research support aid and never as a replacement for independent analysis; AI-generated summaries were not treated as primary evidence.

## 7. References

- [1] S. H. Edwards and M. A. Pérez-Quiñones, “Web-CAT: Automatically Grading Programming Assignments,” *Proceedings of the 13th Annual Conference on Innovation and Technology in Computer Science Education*, 2008. DOI: 10.1145/1384271.1384371. [Paper Link](#)
- [2] P. Ithantola, T. Ahoniemi, V. Karavirta, and O. Seppälä, “Review of Recent Systems for Automatic Assessment of Programming Assignments,” *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, 2010. DOI: 10.1145/1930464.1930480. [Paper Link](#)
- [3] R. Singh, S. Gulwani, and A. Solar-Lezama, “Automated Feedback Generation for Introductory Programming Assignments,” *ACM SIGPLAN Notices (PLDI)*, vol. 48, no. 6, 2013. DOI: 10.1145/2499370.2462195. [Paper Link](#)
- [4] M. Messer, N. C. C. Brown, M. Kölling, and M. Shi, “Automated Grading and Feedback Tools for Programming Education: A Systematic Review,” *ACM Transactions on Computing Education*, vol. 24, no. 1, 2024. DOI: 10.1145/3636515. [Paper Link](#)
- [5] M. Pankiewicz and R. S. Baker, “Large Language Models (GPT) for Automating Feedback on Programming Assignments,” *Proceedings of the 31st International Conference on Computers in Education*, 2023. DOI: 10.58459/icce.2023.950. [Paper Link](#)
- [6] N. Kiesler, D. Lohr, and H. Keuning, “Exploring the Potential of Large Language Models to Generate Formative Programming Feedback,” *IEEE Frontiers in Education Conference*, 2023. DOI: 10.1109/FIE58773.2023.10343457. [Paper Link](#)
- [7] M. Zhu, L. Xu, and B. Ericson, “A Systematic Review of Research on Large Language Models for Computer Programming Education,” 2025. [Paper Link](#)
- [8] J. Hellings and C. Haelermans, “The Effect of Providing Learning Analytics on Student Behaviour and Performance in Programming: A Randomised Controlled Experiment,” *Higher Education*, vol. 83, 2022. DOI: 10.1007/s10734-020-00560-z. [Paper Link](#)
- [9] B.-A. Nguyen, N.-L. Nguyen, and H.-M. Chen, “Utilizing Learning Analytics in an Automated Programming Assessment System to Enhance Software Practice Education,” 2023. DOI: 10.4018/978-1-6684-9527-8.ch017. [Paper Link](#)
- [10] C. Prechelt, M. Malpohl, and M. Philippsen, “Finding Plagiarisms among a Set of Programs with JPlag,” *Journal of Universal Computer Science*, vol. 8, no. 11, 2002. DOI: 10.3217/jucs-008-11-1016. [Paper Link](#)
- [11] O. Karnalim, S. Budi, H. Toba, and M. Joy, “Source Code Plagiarism Detection in Academia with Information Retrieval: Dataset and the Observation,” *Informatics in Education*, vol. 18, no. 2, 2019. DOI: 10.15388/infedu.2019.15. [Paper Link](#)
- [12] S. Guo and X. Lan, “The Effectiveness of Gamification in Programming Education: Evidence from a Meta-analysis,” *Computers and Education: Artificial Intelligence*, vol. 3, 100096, 2022. DOI: 10.1016/j.caeai.2022.100096. [Paper Link](#)